Runge–Kutta Physics-Informed Neural Networks

Motivation from the LHCb Track Extrapolation Problem

G. Scriven

Maastricht University · UHasselt · LHCb

March 2026

Outline

1. The Physics Problem
2. The Speed Problem
3. Why PINNs Failed
4. The RK-PINN Idea
5. The Residual Architecture
6. Literature Solutions
7. The Combined Strategy
8. Roadmap
9. Summary

Track Extrapolation in LHCb

Charged particles traverse the LHCb dipole magnet and are bent by the Lorentz force.

Equations of motion (4 coupled ODEs, z as ind. variable):

$$\frac{dx}{dz} = t_x, \quad \frac{dy}{dz} = t_y$$

$$\frac{dt_x}{dz} = \kappa N \left[t_x t_y B_x - (1+t_x^2) B_y + t_y B_z \right]$$

$$\frac{dt_y}{dz} = \kappa N \left[(1+t_y^2) B_x - t_x t_y B_y - t_x B_z \right]$$

$$\kappa = (q/p) \cdot c_{\text{light}}, \quad N = \sqrt{1 + t_x^2 + t_y^2}$$

The mapping

$f : (x_0, y_0, t_{x,0}, t_{y,0}, q/p, \Delta z) \rightarrow (x_f, y_f, t_{x,f}, t_{y,f})$

6D input $\rightarrow$ 4D output

Challenge

- $\boldsymbol{B}(x, y, z)$ is a **3D tabulated** field
- Non-uniform, non-analytic
- $81 \times 81 \times 146$ grid
- Δz up to 8000 mm

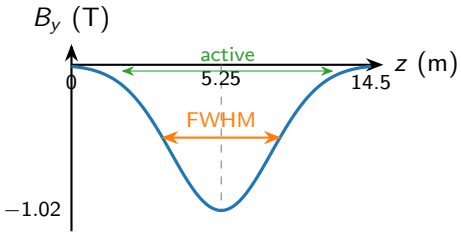
The LHCb Magnetic Field

Field characteristics (from `twodip.rtf`):

- Dominant: B_y (vertical, bends in xz-plane)
- Peak $B_y = -1.02\,\mathrm{T}$ at $z \approx 5250\,\mathrm{mm}$
- FWHM $\approx 3400\,\mathrm{mm}$; active: $z \in [2000, 9400]$
- B_x, B_z: fringe fields ($\sim$10% of B_y)

Gaussian approx. ($B_y \approx B_0 e^{-(z-z_c)^2/2\sigma^2}$):

- RMS error: 20.8 mT (3.2%)
- **Not suitable** for precision training



Key insight

Field is **smooth** and **slowly varying**—ideal for numerical integration.

Why Speed Matters

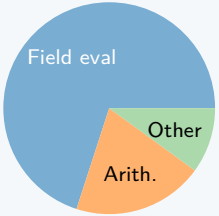
LHCb Upgrade II (Run 5+):

- 40 MHz bunch crossing rate
- Full software trigger (no L0 hardware)
- Track extrapolation: $\mathcal{O}(10^3)$/event

Current C++ RK4 extrapolator:

- CashKarp scheme (6-stage adaptive)
- Per sub-step: **6 field lookups** + interpolation
- Δz=8000 mm, step=5 mm: $\sim$1600$\times$6 = **9600 field evals**
- Total: **7.50 μs per call**

Cost breakdown



$\sim$70% in field lookups

Bottleneck

Field interpolation dominates. An NN replaces **all 9600 lookups** with one forward pass.

Three Architecture Approaches

MLP — Black-box

Direct function approximation:

$$\hat{\boldsymbol{y}} = \mathrm{MLP}(\boldsymbol{x}_0, q/p, \Delta z)$$

- No physics knowledge
- Fastest inference
- Needs abundant data

PINN — Physics loss

MLP + physics regularisation:

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \lambda\, \mathcal{L}_{\text{PDE}}$$

- ODE residual as penalty
- Needs differentiable field
- Slower training (AD)

RK-PINN — Structured

RK4-inspired multi-stage:

$$\hat{\boldsymbol{y}} = \boldsymbol{y}_0 + \sum_k w_k\, \Delta \boldsymbol{s}_k$$

- Physics in architecture
- IC by construction
- Learnable RK weights

Each approach represents a different level of physics integration:
none (MLP) $\rightarrow$ **soft constraint** (PINN) $\rightarrow$ **structural bias** (RK-PINN)

Benchmark Results: MLPs Win on Speed

Model	Type	Params	Time (μs)	Speedup	Error
mlp_single	MLP	18k	**0.83**	**9.0**×	65 μm
mlp_tiny	MLP	4.9k	1.12	6.7×	53 μm
mlp_small	MLP	17.9k	1.22	6.1×	53 μm
mlp_shallow	MLP	100k	1.28	5.9×	33 μm
mlp_medium	MLP	101k	2.17	3.5×	37 μm
pinn_medium	PINN	101k	2.46	3.0×	593 mm
rkpinn_tiny	RK-PINN	—	3.91	1.9×	$> 10^5$ μm
rkpinn_medium	RK-PINN	202k	5.59	1.3×	2 km
Kisel	C++	—	1.50	5.0×	39.8 mm
Reference RK4	C++	—	7.50	1.0×	0 (ref.)

Physics-informed models were slower AND catastrophically wrong.

But was the failure fundamental, or fixable?

PINN Failure — Root Cause Analysis

Bug: `forward()` hardcoded `z_frac = 1.0`

1. Network only trained at **one** z-value
2. Collocation points evaluated **untrained** regime
3. Output independent of $z \Rightarrow$ IC violated
4. Physics loss was **meaningless noise**

Additional issues:

- Gaussian field $\neq$ real field (3.2% mismatch)
- AD through Lorentz force: $3\times$ training cost

Literature confirms this

Krishnapriyan et al. (NeurIPS 2021):

- PINNs fail on *slightly* complex problems
- Soft PDE loss $\rightarrow$ ill-conditioned landscape
- Fixes: curriculum training, domain decomposition

Our 4-ODE system

Coupled ODEs + non-uniform 3D field = exactly the complexity regime where PINNs fail.

Three Distinct Problems to Solve

1. IC Violation
Network ignores z;
output constant for all z_{frac}

↓

Fix: Residual architecture
$\boldsymbol{y} = \boldsymbol{y}_0 + z \cdot \Delta \boldsymbol{y}$

2. Field Mismatch
Gaussian $B_y(z)$ in
physics loss $\neq$ real field

↓

Fix: FD-based physics loss
Use tabulated field directly

3. Speed Overhead
AD through Lorentz force:
$3\times$ slower training

↓

Fix: RK structure as
inductive bias, not AD

The RK-PINN architecture addresses all three.

From RK4 to RK-PINN: The Classical Update

Given $\frac{d\boldsymbol{y}}{dz} = \boldsymbol{f}(\boldsymbol{y}, z)$, the classical RK4 method advances from z_n to $z_n + h$:

Four stage evaluations:

$$\boldsymbol{k}_1 = h\,\boldsymbol{f}(\boldsymbol{y}_n,\ z_n)$$

$$\boldsymbol{k}_2 = h\,\boldsymbol{f}\left(\boldsymbol{y}_n + \tfrac{1}{2}\boldsymbol{k}_1,\ z_n + \tfrac{h}{2}\right)$$

$$\boldsymbol{k}_3 = h\,\boldsymbol{f}\left(\boldsymbol{y}_n + \tfrac{1}{2}\boldsymbol{k}_2,\ z_n + \tfrac{h}{2}\right)$$

$$\boldsymbol{k}_4 = h\,\boldsymbol{f}(\boldsymbol{y}_n + \boldsymbol{k}_3,\ z_n + h)$$

Weighted combination:

$$\boldsymbol{y}_{n+1} = \boldsymbol{y}_n + \tfrac{1}{6}\boldsymbol{k}_1 + \tfrac{1}{3}\boldsymbol{k}_2 + \tfrac{1}{3}\boldsymbol{k}_3 + \tfrac{1}{6}\boldsymbol{k}_4$$

Key structural observations:

- Each $\boldsymbol{k}_i$ = state increment at a z-fraction
- Output = initial state + weighted sum
- Weights $[\frac{1}{6}, \frac{1}{3}, \frac{1}{3}, \frac{1}{6}]$ from Butcher tableau

Insight: This structure — initial state plus a weighted sum of increments at intermediate points — can be *directly encoded* as a neural network architecture.

From RK4 to RK-PINN: The Neural Analogue

Replace the analytic derivative $\boldsymbol{f}(\boldsymbol{y}, z)$ with learned stage increments:

Shared feature extraction:

$$\boldsymbol{h} = \text{Backbone}(\boldsymbol{x}_0, q/p, \Delta z)$$

Per-stage predictions:

$$\Delta \boldsymbol{s}_k = \text{Head}_k(\boldsymbol{h},\ z_k), \quad k = 1, \ldots, 4$$

where $z_k = z_0 + c_k \cdot \Delta z, \quad c_k \in \{\frac{1}{4}, \frac{1}{2}, \frac{3}{4}, 1\}$.

Learnable RK combination:

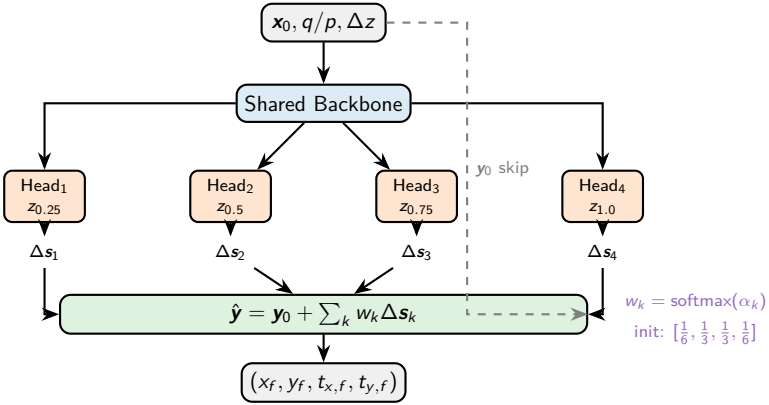
$$\hat{\boldsymbol{y}} = \boldsymbol{y}_0 + \sum_{k=1}^{4} w_k \, \Delta \boldsymbol{s}_k$$

$w_k = \text{softmax}(\alpha_k)$, initialised to $[\frac{1}{6}, \frac{1}{3}, \frac{1}{3}, \frac{1}{6}]$.

- Starts as RK4; free to deviate during training
- Skip connection $\boldsymbol{y}_0 \to$ output guarantees IC

The RK-PINN **preserves the RK4 decomposition** but replaces numerical derivative evaluations with learned functions sharing a common backbone.

RK-PINN Architecture Diagram



Why RK Structure Helps Speed

C++ RK4 (CashKarp):

- 6-stage adaptive scheme
- $\Delta z = 8000\,\text{mm}$, step $= 5\,\text{mm}$
- 1600 sub-steps $\times$ 6 $=$ **9600 field evals**
- Each: trilinear lookup in $81^2 \times 146$ grid
- Cost: **7.50** μ**s**

Pure MLP: single forward pass, no field lookups.
Cost: **0.83–2.17** μ**s**. Learns mapping as black box.

RK-PINN:

- 4 stage heads (parallel-capable)
- Shared backbone: computed **once**
- **No field lookups at inference**
- Physics encoded in *structure* not loss

Speed comparison

Method	**Field evals**
C++ CashKarp	9600
PINN (10 collocation)	10 *(train only)*
RK-PINN / MLP	**0**

Residual Architecture — Guaranteeing Initial Conditions

Constraint: at $z{=}z_0$, $\hat{\boldsymbol{y}}(z_0) = \boldsymbol{y}_0$.

Naïve: $\hat{\boldsymbol{y}} = \mathrm{NN}(\boldsymbol{x}_0, q/p, \Delta z)$ — no z_f, IC not enforced. ✗

Residual: parameterise *deviation*:

$\hat{\boldsymbol{y}}(z) = \boldsymbol{y}_0 + z_f \cdot \Delta \boldsymbol{y}_{\mathrm{NN}}$

$z_f{=}0 \Rightarrow \hat{\boldsymbol{y}}{=}\boldsymbol{y}_0$ ✓; $z_f{=}1 \Rightarrow$ full correction.

Impact

	Naïve	Residual
IC err.	2758 mm	**0.0 mm**
Pos. err.	593 mm	28 μm

Explicit form ($z_f \equiv z_{\mathrm{frac}}$):

Positions (straight-line + deflection):

$$\hat{x} = x_0 + t_{x,0}\Delta z\, z_f + z_f \cdot \delta x_{\mathrm{NN}}$$
$$\hat{y} = y_0 + t_{y,0}\Delta z\, z_f + z_f \cdot \delta y_{\mathrm{NN}}$$

Slopes (constant + curvature):

$$\hat{t}_x = t_{x,0} + z_f \cdot \delta t_{x,\mathrm{NN}}$$
$$\hat{t}_y = t_{y,0} + z_f \cdot \delta t_{y,\mathrm{NN}}$$

Network outputs 4 corrections $(\delta x, \delta y, \delta t_x, \delta t_y)$; residual skip connection **guarantees** $\hat{\boldsymbol{y}}(z_0) = \boldsymbol{y}_0$.

Paper 1: Finite Difference PINNs

"Physics-informed deep learning based on the finite difference method"

Core idea: Replace AD with finite differences for the physics loss.

$$\left.\frac{\partial \hat{u}}{\partial z}\right|_{z_i} \approx \frac{\hat{u}(z_i+\delta)-\hat{u}(z_i-\delta)}{2\delta}$$

Why this matters for us:

1. AD needs differentiable field $\Rightarrow$ forced Gaussian
2. FD only needs evaluations $\Rightarrow$ use `twodip.rtf`
3. No computational graph $\Rightarrow$ lower memory
4. FD $\approx$ MLP training speed (vs. $3\times$ for AD)

Field model comparison

	AD	**FD**
Differentiable?	Required	Not needed
Field model	Gaussian	Any
Mismatch	3.2%	**0%**
Memory	High	Low
Speed	$3\times$ MLP	$\sim 1.2\times$

For RK-PINN

FD physics loss at each RK stage with the real field map = **physically correct** training.

Paper 2: Data-Free PINNs

Misyris et al. — "Learning without Data," IEEE SmartGridComm (2021)

Core idea: Train with **zero data**, physics loss only.

$$\mathcal{L} = \lambda_{\text{PDE}}\,\mathcal{L}_{\text{PDE}} + \lambda_{\text{IC}}\,\mathcal{L}_{\text{IC}}$$

Their result:

- Solved power grid DAE system (no data)
- Orders-of-magnitude speedup
- Generalises across IC families

For us:

- Pure data-free unlikely (6D, complex field)
- **Hybrid:** 1–5M samples + physics loss

Data requirements

	Initial	**Hybrid**
Training data	50M	1–5M
Gen. time	Hours	Minutes
Physics loss	Broken	Correct

Speed implication

Less training data $\Rightarrow$ faster iteration cycles.
Physics loss compensates for reduced data coverage.

Paper 3: Temporal Domain Decomposition with DeepONets

"Long-time integration of parametric evolution equations with physics-informed DeepONets"

Problem: PINNs fail for long time horizons (spectral bias).

Solution: Split into short windows, chain predictions.

Direct relevance:

- Initial study: fixed $\Delta z = 8000\,\mathrm{mm}$ (monolithic)
- **Short-step chaining** (e.g. $4 \times 2000\,\mathrm{mm}$) enables variable step sizes
- Compatible with Kalman filter (sequential)

Speed estimate

- Model for $\Delta z = 2000\,\mathrm{mm}$
- $\sim 0.5\,\mu\mathrm{s}/\mathrm{step} \times 4 = 2.0\,\mu\mathrm{s}$
- Still $\leq$ C++ RK4 ($7.50\,\mu\mathrm{s}$)

Operator learning

DeepONet learns the *solution operator*: $\mathcal{G} : \boldsymbol{s}_0 \mapsto \boldsymbol{s}(\Delta z)$. One network, all initial states & momenta.

Paper 4: RK-PINNs in the Literature

"Parameter estimation and modeling of nonlinear dynamical systems based on RK-PINN"

Literature RK-PINN:

- Multi-stage with Butcher tableau weights
- **Full PDE residual** at each stage
- IC satisfaction via residual form
- Forward + inverse problems

vs. our initial RK-PINN:

✗ IC not guaranteed
✗ Gaussian field in physics loss
✓ Learnable softmax weights
✓ Shared backbone

Inverse problem capability

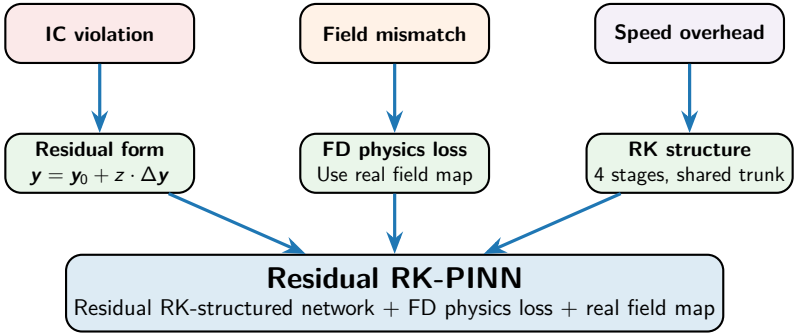
RK-PINN could **simultaneously**:

1. Predict trajectory (forward)
2. Estimate q/p (inverse)
3. Refine field map (calibration)

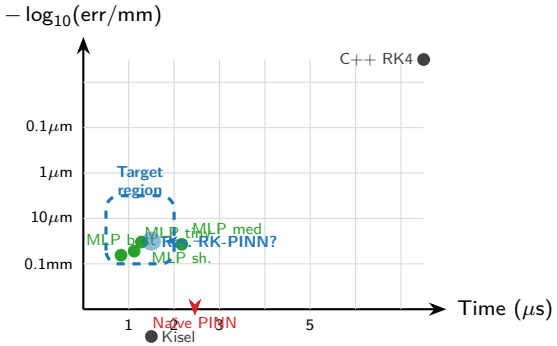
What we need to fix

1. Residual PINN formulation
2. Full Lorentz residual at each stage
3. FD-based physics loss (real field)

Putting It Together: Residual RK-PINN



Speed-Accuracy Trade-off: Target Region



Target specs

- Inference: $\leq 2.0\,\mu$s
- Accuracy: $\leq 100\,\mu$m
- Physics-consistent trajectories
- Variable Δz support

Expected performance

- RK-PINN *med*: $5.59\,\mu$s
- Backbone shared, stage heads differ
- **Tiny:** $\sim$1.5–2.0 μs
- Physics $\Rightarrow$ better accuracy/param

Implementation Priority

P1 **Residual architecture fix** — prerequisite for everything

- $\boldsymbol{y}(z) = \boldsymbol{y}_0 + z_{\text{frac}} \cdot \text{MLP}(\cdots)$; guarantees IC by construction

P2 **FD-based physics loss** with real field map

- Replace AD with $(\hat{u}(z+\delta) - \hat{u}(z-\delta))/2\delta$; use `twodip.rtf` directly
- Training speed: $\sim$MLP speed (vs. $3\times$ for AD)

P3 **Residual RK-PINN** with short-step chaining

- Train on $\Delta z = 1000$–$2000\,\text{mm}$; chain 4–8 steps
- $\sim 0.5\,\mu\text{s/step}$; total 2–4 μs (competitive with C++)

P4 **Inverse mode** — exploratory

- Estimate q/p from track hits; field map calibration

Summary

1. **The problem is well-suited for RK-PINNs:**
   - Lorentz force ODEs have RK4 integration as natural structure
   - Field is smooth, slowly varying, tabulated on regular grid
   - Speed critical: 9600 field evaluations $\rightarrow$ single forward pass
2. **Initial failures were architectural, not fundamental:**
   - IC violation $\rightarrow$ residual fix; field mismatch $\rightarrow$ FD physics loss
   - Literature confirms: PINNs fail on complex ODEs unless carefully designed
3. **Literature provides a clear path:**
   - FD-PINNs, domain decomposition, RK-structure, inverse capability
4. **Target:** Residual RK-PINN at $\leq 2\,\mu$s, $\leq 100\,\mu$m accuracy, physics-consistent trajectories.

References I

- M. Raissi, P. Perdikaris, G.E. Karniadakis, "Physics-informed neural networks," *J. Comput. Phys.* **378**, 686 (2019).
- "Physics-informed deep learning based on the finite difference method for efficient and accurate numerical solution of PDEs."
- G.S. Misyris, A. Stiasny, S. Chatzivasileiadis, "Learning without Data: PINNs for Fast Time-Domain Simulation," *IEEE SmartGridComm* (2021).
- "Long-time integration of parametric evolution equations with physics-informed DeepONets."
- "Parameter estimation and modeling of nonlinear dynamical systems based on RK-PINN."
- A.S. Krishnapriyan et al., "Characterizing possible failure modes in PINNs," *NeurIPS* (2021).
- L. Lu, P. Jin, G.E. Karniadakis, "DeepONet: Learning nonlinear operators," *Nature Mach. Intell.* **3**, 218 (2021).

References II

Z. Li et al., "Fourier Neural Operator for Parametric PDEs," *ICLR* (2021).

Thank you

Questions?

George.Scriven@uhasselt.be
George.William.Scriven@cern.ch